

Procesador Paralelo de Imágenes con Erlang y Scheme

J. Pablo Valverde Barrantes
Kendal Josue Chacón Chaves

23 de setiembre de 2026

Índice

1. Introducción	3
2. Arquitectura del sistema	3
3. Formato de imagen	4
4. División de la imagen	4
4.1. Halo	5
5. Modelo de concurrencia	5
6. Comunicación entre Erlang y Scheme	6
7. Protocolo de comunicación	6
8. Implementación de los filtros	7
8.1. Gaussian Blur	7
8.2. Escala de grises	8
8.3. Inversión de colores	8
8.4. Ajuste de brillo	8
8.5. Threshold	8
8.6. Sharpen	9
8.7. Sobel	9
9. Manejo de los bordes	9

10.Manejo de errores y tolerancia a fallos	10
11.Reconstrucción de la imagen	11
12.Ejecución del programa	11
13.Metodología experimental	11
14.Evaluación del rendimiento	12
14.1. Tiempo de ejecución	12
14.2. Speedup	13
14.3. Eficiencia	15
14.4. Speedup superlineal	16
14.5. Rendimientos decrecientes	16
14.6. Comportamiento de la imagen p3	17
14.7. Estabilidad de las mediciones	18
15.Limitaciones del sistema	18
16.Conclusiones	19

1. Introducción

En este proyecto se desarrolló un procesador paralelo de imágenes utilizando Erlang y Scheme. La idea principal consiste en separar las responsabilidades entre ambos lenguajes: Erlang se encarga de la lectura de la imagen, la división del trabajo, la creación de procesos concurrentes, la comunicación y la reconstrucción final; mientras que Scheme se utiliza exclusivamente para realizar las transformaciones sobre los píxeles.

El programa trabaja con imágenes en formato PPM P3 y permite aplicar como filtro principal un desenfoque gaussiano (*Gaussian Blur*). Además, se implementaron filtros adicionales como escala de grises, inversión de colores, ajuste de brillo, *threshold*, *sharpen* y detección de bordes mediante Sobel.

El objetivo principal no fue solamente aplicar los filtros, sino analizar cómo cambia el rendimiento al dividir una misma imagen entre varios procesos Erlang. Para esto se realizaron pruebas utilizando 1, 2, 4 y 8 procesos y se calcularon las métricas de tiempo de ejecución, *speedup*, eficiencia y, adicionalmente, throughput.

2. Arquitectura del sistema

La arquitectura del sistema utiliza a Erlang como coordinador general de la ejecución. Scheme no administra procesos ni reconstruye imágenes completas, sino que recibe una región, aplica el filtro solicitado y devuelve solamente las filas correspondientes al bloque asignado.

El flujo general del sistema es el siguiente:

1. Erlang recibe el archivo PPM de entrada.
2. Se lee el archivo y se obtienen sus dimensiones.
3. La imagen se representa como una lista de filas de píxeles RGB.
4. Erlang divide la imagen en regiones.
5. Se agregan las regiones de contexto o *halo* necesarias.
6. Se crea un proceso Erlang por cada región.
7. Cada trabajador ejecuta una instancia independiente de Scheme.
8. Scheme aplica el filtro correspondiente.
9. Cada trabajador devuelve su resultado al coordinador mediante paso de mensajes.
10. Erlang ordena las regiones según su posición original.
11. Las regiones se concatenan para reconstruir la imagen.
12. Finalmente se genera el archivo PPM de salida.

Esta separación permite que Scheme se concentre en el procesamiento funcional de los píxeles y que Erlang se encargue completamente de la concurrencia y coordinación.

3. Formato de imagen

El sistema utiliza imágenes PPM en formato P3. Este formato almacena los valores de los píxeles como texto y permite trabajar con ellos sin depender de bibliotecas externas de procesamiento de imágenes.

Una imagen PPM P3 contiene inicialmente:

```
P3
ancho alto
255
R G B
R G B
...
```

Cada píxel se representa mediante tres componentes:

$$(R, G, B)$$

donde cada valor se encuentra entre 0 y 255.

El lector implementado en Erlang obtiene el ancho, el alto y el valor máximo de color, agrupa los valores RGB en píxeles y finalmente construye las filas de la imagen.

El sistema también elimina comentarios presentes en archivos PPM antes de procesar sus valores.

4. División de la imagen

Para distribuir el trabajo se decidió dividir la imagen en franjas horizontales.

Esta decisión simplifica el cálculo de las regiones y del *halo*, ya que cada proceso recibe un conjunto consecutivo de filas.

Si una imagen contiene H filas y se utilizan p procesos, inicialmente se calcula:

$$q = \left\lfloor \frac{H}{p} \right\rfloor$$

y:

$$r = H \bmod p$$

Cada proceso recibe aproximadamente q filas. Las r filas sobrantes se distribuyen entre las primeras regiones, agregando una fila adicional a cada una hasta consumir el resto.

De esta manera la carga queda distribuida de la forma más uniforme posible.

Además, si se solicitan más procesos que filas tiene la imagen, el número real de procesos se limita a la cantidad de filas existentes.

4.1. Halo

Los filtros basados en convolución presentan un problema en los límites entre regiones. Por ejemplo, un kernel 3×3 necesita conocer la fila anterior y la fila siguiente del píxel que se está procesando.

Para solucionar esto se utiliza un *halo*.

Para un kernel de tamaño k , el tamaño del halo utilizado es:

$$h = \left\lceil \frac{k}{2} \right\rceil$$

Por lo tanto, para el Gaussian Blur obligatorio de 3×3 :

$$h = 1$$

Cada región interna recibe una fila adicional superior y otra inferior. Estas filas se utilizan como contexto para calcular correctamente los píxeles cercanos a los límites de la región.

Scheme procesa la región completa incluyendo el halo, pero antes de devolver el resultado elimina las filas de contexto. De esta manera cada proceso devuelve únicamente el bloque que realmente le fue asignado.

Para un kernel 5×5 , que también es soportado por la implementación, el halo aumenta a dos filas.

5. Modelo de concurrencia

La concurrencia se implementa utilizando el modelo de procesos de Erlang.

Después de dividir la imagen, la función encargada de crear los trabajadores utiliza `spawn` para generar un proceso Erlang independiente para cada región.

Conceptualmente:

```
Coordinador
|
+---- Trabajador 1 ---> Scheme 1
|
+---- Trabajador 2 ---> Scheme 2
|
+---- Trabajador 3 ---> Scheme 3
|
+---- ...
```

Cada trabajador procesa solamente su región y posteriormente envía un mensaje al coordinador.

Los mensajes de éxito poseen la información necesaria para identificar la región y reconstruir posteriormente la imagen. Si ocurre un fallo, el proceso trabajador envía en su lugar un mensaje de error.

Debido a que los trabajadores son procesos Erlang independientes, las regiones pueden ejecutarse de manera concurrente. La ejecución no consiste en una secuencia de llamadas a Scheme, sino que cada trabajador inicia su propia instancia.

6. Comunicación entre Erlang y Scheme

Para ejecutar Scheme desde Erlang se utiliza `open_port`. Esta funcionalidad permite lanzar un proceso externo y recibir información acerca de su ejecución y su código de salida.

Cada región utiliza tres archivos temporales:

1. Un archivo de entrada que contiene la solicitud para Scheme.
2. Un archivo *wrapper* que carga el archivo principal del procesador y ejecuta la solicitud.
3. Un archivo de salida donde Scheme escribe la región procesada.

Los archivos se generan con identificadores únicos para evitar conflictos entre los diferentes procesos concurrentes.

Después de terminar el procesamiento de una región, los tres archivos temporales son eliminados.

El uso de archivos temporales simplificó la integración con las funciones de entrada y salida disponibles en Scheme, aunque introduce un costo adicional de lectura, escritura y creación de archivos que también influye en el rendimiento del sistema.

7. Protocolo de comunicación

La comunicación entre Erlang y Scheme utiliza una solicitud con la siguiente estructura conceptual:

```
(request filtro
tamano-kernel
parametro
filas-halo-arriba
alto-bloque
region-con-halo)
```

Los campos utilizados son:

- **filtro**: transformación que debe aplicarse.
- **tamano-kernel**: tamaño del kernel cuando el filtro lo necesita.
- **parametro**: valor adicional utilizado por filtros como brillo o threshold.
- **filas-halo-arriba**: cantidad de filas de contexto antes del bloque real.
- **alto-bloque**: cantidad de filas que pertenecen realmente a la región.

- **region-con-halo:** matriz de píxeles que Scheme debe procesar.

Cada píxel se transmite utilizando una lista:

$$(R\ G\ B)$$

Scheme aplica el filtro a toda la región recibida. Posteriormente utiliza la cantidad de filas de halo y el alto original del bloque para recortar el resultado y devolver únicamente las filas correspondientes a la región real.

La respuesta se escribe como valores RGB en texto plano. Erlang vuelve a agrupar estos valores en píxeles y verifica que la cantidad recibida coincida con el ancho de la imagen multiplicado por el alto esperado del bloque.

8. Implementación de los filtros

Todos los filtros de procesamiento de imágenes se encuentran implementados en Scheme.

La implementación se realizó utilizando principalmente funciones, recursión, listas, `lambda`, `map` y `cond`, manteniendo el procesamiento de imágenes dentro del paradigma funcional.

8.1. Gaussian Blur

El filtro principal del proyecto es el desenfoque gaussiano.

Para el kernel obligatorio de 3×3 se utiliza:

$$G = \frac{1}{16} \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Para cada píxel se obtiene una vecindad de 3×3 . Cada píxel vecino se multiplica por el peso correspondiente del kernel.

Para un canal de color puede expresarse como:

$$I'(x, y) = \frac{\sum_{i=-1}^1 \sum_{j=-1}^1 G(i, j) I(x + i, y + j)}{16}$$

El mismo procedimiento se aplica independientemente para los componentes rojo, verde y azul.

También se implementó como extensión un kernel gaussiano de 5×5 .

8.2. Escala de grises

La intensidad se calcula utilizando una aproximación ponderada:

$$Y = \frac{30R + 59G + 11B}{100}$$

El nuevo píxel es:

$$(Y, Y, Y)$$

8.3. Inversión de colores

Cada componente se transforma mediante:

$$R' = 255 - R$$

$$G' = 255 - G$$

$$B' = 255 - B$$

8.4. Ajuste de brillo

Para una cantidad d :

$$C' = \text{clamp}(C + d, 0, 255)$$

donde C representa cualquiera de los tres canales RGB.

La función `clamp` evita que el resultado quede fuera del intervalo permitido.

8.5. Threshold

Primero se calcula la intensidad utilizando la misma expresión de escala de grises.

Luego, para un umbral u :

$$P' = \begin{cases} (255, 255, 255), & Y \geq u \\ (0, 0, 0), & Y < u \end{cases}$$

8.6. Sharpen

Se utiliza el kernel:

$$\begin{pmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{pmatrix}$$

Este kernel aumenta la diferencia entre un píxel y sus vecinos, resaltando detalles y bordes de la imagen.

8.7. Sobel

Para la detección de bordes se utilizan dos kernels:

$$G_x = \begin{pmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{pmatrix}$$

y:

$$G_y = \begin{pmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{pmatrix}$$

Después de obtener ambos gradientes, la implementación aproxima la magnitud mediante:

$$|G_x| + |G_y|$$

en lugar de calcular la raíz cuadrada de la suma de cuadrados. El resultado se limita nuevamente al intervalo $[0, 255]$.

9. Manejo de los bordes

Existen dos tipos de límites que deben considerarse.

El primero corresponde a los límites entre regiones. Estos se solucionan utilizando el mecanismo de halo descrito anteriormente.

El segundo corresponde a los límites reales de la imagen, donde no existe una región vecina de la cual obtener información.

Para estos casos se utiliza la estrategia de **repetir el píxel más cercano**.

Si durante la convolución se solicita un índice menor que cero, se utiliza el índice cero. Si se solicita una posición superior al límite, se utiliza la última posición válida.

Por ejemplo:

$$I(-1, y) = I(0, y)$$

y:

$$I(H, y) = I(H - 1, y)$$

Esto permite aplicar el mismo algoritmo de convolución a todos los píxeles sin utilizar casos especiales para cada esquina o borde.

10. Manejo de errores y tolerancia a fallos

El sistema implementa diferentes mecanismos para evitar que un fallo produzca silenciosamente una imagen incorrecta.

Entre los errores que pueden detectarse se encuentran:

- Archivo de entrada inexistente.
- Formato PPM inválido.
- Cantidad incorrecta de píxeles.
- Parámetros de filtro inválidos.
- Falta de un intérprete Scheme disponible.
- Error al crear los archivos temporales.
- Terminación incorrecta de Scheme.
- Tiempo máximo de ejecución excedido.
- Archivo de salida de Scheme inexistente.
- Cantidad incompleta o inválida de píxeles retornados por Scheme.

Cuando una instancia de Scheme termina con un código de salida distinto de cero, Erlang registra el fallo y lo devuelve al trabajador.

Además, se implementó un reintento automático. Si una región falla durante el primer intento, Erlang ejecuta nuevamente Scheme para esa misma región. Si el segundo intento también falla, el error se comunica al coordinador.

El tiempo máximo predeterminado para una instancia de Scheme es de 300000 ms. Este valor puede modificarse mediante la variable de entorno `SCHEME_TIMEOUT_MS`.

11. Reconstrucción de la imagen

Los procesos concurrentes no necesariamente terminan en el mismo orden en que fueron creados.

Por este motivo, cada región posee un índice.

Cuando el coordinador recibe todos los resultados, las regiones se ordenan utilizando este índice. Posteriormente sus filas se concatenan para recuperar el orden original.

Como Scheme devuelve únicamente las filas pertenecientes al bloque real y no devuelve las filas del halo, no se producen filas duplicadas durante la reconstrucción.

12. Ejecución del programa

Una ejecución del Gaussian Blur 3×3 utilizando cuatro procesos puede realizarse mediante:

```
./image_processor entrada.ppm salida.ppm 4 gaussian 3
```

Algunos ejemplos de filtros adicionales son:

```
./image_processor entrada.ppm salida.ppm 4 grayscale
./image_processor entrada.ppm salida.ppm 4 invert
./image_processor entrada.ppm salida.ppm 4 brightness 30
./image_processor entrada.ppm salida.ppm 4 threshold 128
./image_processor entrada.ppm salida.ppm 4 sharpen
./image_processor entrada.ppm salida.ppm 4 sobel
```

El modo de benchmark puede ejecutarse mediante:

```
./image_processor --benchmark entrada.ppm resultados gaussian 3
```

Este modo realiza automáticamente las pruebas con 1, 2, 4 y 8 procesos y genera un archivo CSV.

13. Metodología experimental

Para analizar el rendimiento se utilizó el Gaussian Blur de 3×3 , debido a que corresponde al filtro principal solicitado para el proyecto.

Se utilizaron cinco imágenes PPM de distintos tamaños.

Cuadro 1: Imágenes utilizadas para las pruebas

Imagen	Ancho	Alto	Píxeles	Tamaño aprox. (MB)
p1	501	890	445890	4.73
p2	501	890	445890	4.64
p3	1181	890	1051090	10.95
p4	501	890	445890	4.79
p5	514	890	457460	4.70

Para cada imagen se utilizaron las configuraciones:

$$p \in \{1, 2, 4, 8\}$$

Cada configuración se ejecutó tres veces.

Por lo tanto, el experimento principal contiene:

$$5 \times 4 \times 3 = 60$$

mediciones.

El tiempo medido corresponde al procesamiento completo de la imagen. Esto incluye la lectura del PPM, división de la imagen, creación de procesos Erlang, ejecución de Scheme, comunicación, reconstrucción de regiones y escritura de la imagen resultante.

Al realizar tres repeticiones se reduce el efecto de pequeñas variaciones producidas por el sistema operativo, planificación de procesos o carga temporal del equipo.

Para el análisis se utilizó principalmente el tiempo promedio de las tres ejecuciones.

14. Evaluación del rendimiento

14.1. Tiempo de ejecución

Para una cantidad de procesos p , se define:

$$T_p$$

como el tiempo necesario para completar el procesamiento de una imagen utilizando p procesos.

Los tiempos promedio obtenidos fueron:

Cuadro 2: Tiempo promedio de ejecución en segundos

Imagen	T_1	T_2	T_4	T_8
p1	65.167	25.318	12.198	10.214
p2	63.575	24.932	12.639	9.904
p3	215.646	90.717	48.043	39.471
p4	63.401	24.588	12.451	9.541
p5	65.196	25.493	12.843	10.001

En todas las imágenes se observa una reducción considerable del tiempo al aumentar la cantidad de procesos.

Por ejemplo, para la imagen p4 se pasó de:

$$T_1 = 63,401 \text{ s}$$

a:

$$T_8 = 9,541 \text{ s}$$

Esto representa una reducción aproximada del 84.95 % en el tiempo total.

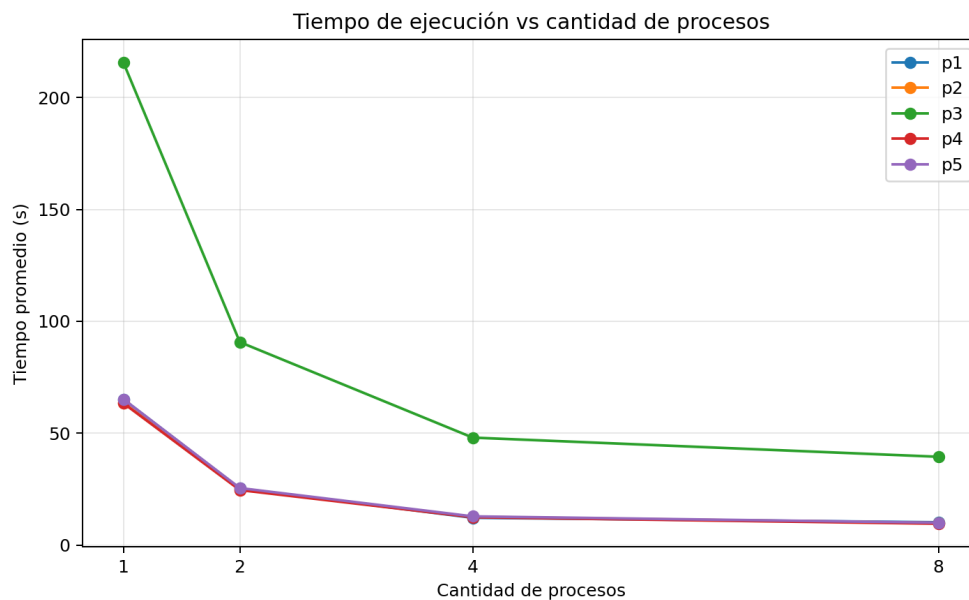


Figura 1: Tiempo de ejecución según la cantidad de procesos.

14.2. Speedup

El speedup se calcula mediante:

$$S_p = \frac{T_1}{T_p}$$

El speedup ideal sería:

$$S_p = p$$

ya que idealmente duplicar la cantidad de procesos debería reducir el tiempo a la mitad.

Los resultados obtenidos utilizando ocho procesos fueron:

Cuadro 3: Speedup y eficiencia con ocho procesos

Imagen	S_8	E_8
p1	6.403	0.800
p2	6.419	0.802
p3	5.464	0.683
p4	6.645	0.831
p5	6.519	0.815

Ninguna de las imágenes alcanzó el speedup ideal de 8 con ocho procesos, aunque todas obtuvieron una mejora considerable con respecto a la ejecución secuencial.

El mejor resultado entre estas pruebas se obtuvo para p4, con:

$$S_8 = 6,645$$

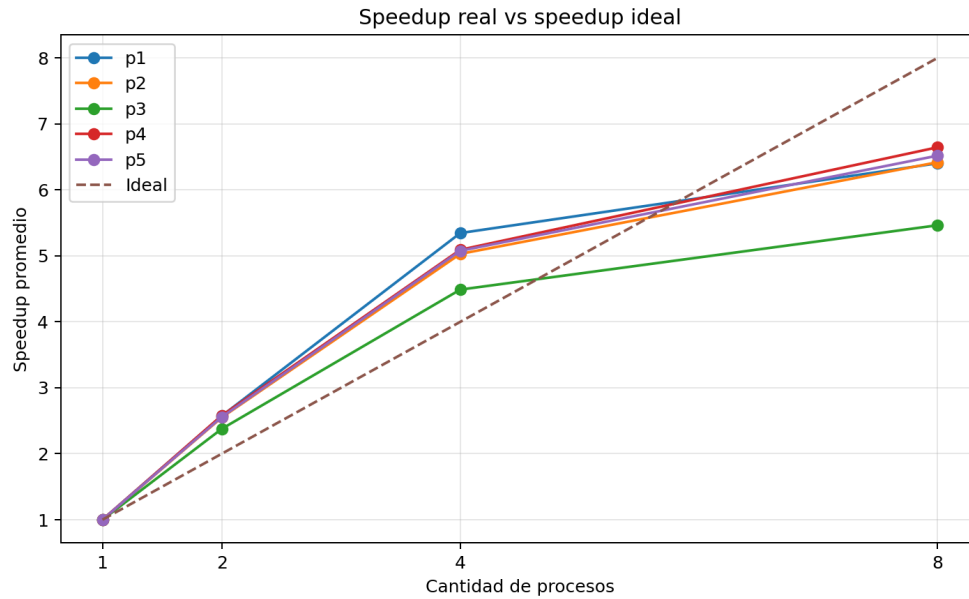


Figura 2: Speedup obtenido comparado con el speedup ideal.

14.3. Eficiencia

La eficiencia se calcula mediante:

$$E_p = \frac{S_p}{p}$$

Una eficiencia igual a 1 representa un aprovechamiento ideal de todos los procesos.

Promediando los resultados de las cinco imágenes se obtuvo:

Cuadro 4: Speedup y eficiencia promedio

Procesos	Speedup promedio	Eficiencia promedio
1	1.000	1.000
2	2.527	1.264
4	5.007	1.252
8	6.290	0.786

Con ocho procesos la eficiencia promedio fue aproximadamente:

$$E_8 = 0,786$$

lo que significa que, en promedio, el sistema alcanzó aproximadamente un 78.6 % del speedup lineal ideal para esta configuración.

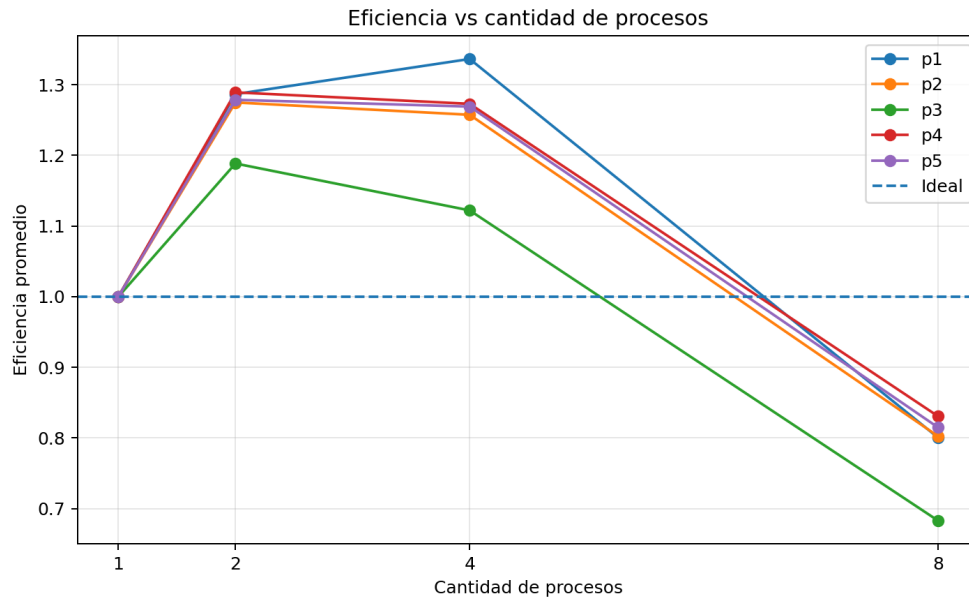


Figura 3: Eficiencia según la cantidad de procesos.

14.4. Speedup superlineal

Un resultado interesante aparece con dos y cuatro procesos, donde varias pruebas presentan una eficiencia mayor que 1.

Por ejemplo, para p1:

$$S_4 = 5,346$$

por lo que:

$$E_4 = \frac{5,346}{4} = 1,336$$

Este comportamiento se conoce como *speedup superlineal*.

14.5. Rendimientos decrecientes

Aunque pasar de un proceso a dos y posteriormente a cuatro produce reducciones importantes, el cambio de cuatro a ocho procesos muestra un beneficio menor.

La reducción de tiempo entre cuatro y ocho procesos fue aproximadamente:

- p1: 16.26 %.
- p2: 21.64 %.
- p3: 17.84 %.
- p4: 23.37 %.
- p5: 22.13 %.

Esto indica que aumentar el número de procesos continúa mejorando el tiempo absoluto, pero el beneficio marginal disminuye.

Entre las principales causas se encuentran:

- Creación de procesos Erlang.
- Creación de una instancia independiente de Scheme por región.
- Comunicación mediante paso de mensajes.
- Conversión de los píxeles de Erlang a representación textual de Scheme.
- Escritura de archivos temporales.
- Lectura de los resultados generados por Scheme.
- Copia de regiones y halos.

- Serialización y deserialización de datos.
- División inicial de la imagen.
- Ordenamiento y reconstrucción de las regiones.
- Escritura del archivo PPM final.
- Competencia entre procesos por CPU, caché y memoria.

Estas operaciones no disminuyen necesariamente en la misma proporción que aumenta la cantidad de procesos y algunas incluso aumentan al crear más regiones.

14.6. Comportamiento de la imagen p3

La imagen p3 contiene:

1051090

píxeles, más del doble que las imágenes p1, p2 y p4.

Su tiempo con un proceso fue:

215,646 s

mientras que con ocho procesos disminuyó hasta:

39,471 s

obteniendo:

$S_8 = 5,464$

y:

$E_8 = 0,683$

Aunque la mejora sigue siendo importante, la eficiencia con ocho procesos es menor que en el resto de las imágenes.

Esto puede relacionarse con el mayor volumen de datos que debe copiarse, serializarse y transferirse entre Erlang y Scheme, además del aumento del trabajo de lectura, escritura y reconstrucción.

También existe un mayor uso de memoria y caché debido al tamaño de las estructuras de listas utilizadas durante el procesamiento.

14.7. Estabilidad de las mediciones

La utilización de tres repeticiones permitió observar que la mayoría de las configuraciones presentan resultados bastante estables.

Las imágenes p2, p3, p4 y p5 mostraron variaciones pequeñas entre repeticiones. La principal excepción fue p1, principalmente con ocho procesos, donde se observó una variación mayor.

Estas diferencias pueden producirse por procesos del sistema operativo, planificación de tareas, utilización momentánea de CPU o efectos de caché.

Por esta razón se utilizaron tiempos promedio en lugar de depender de una única ejecución.

15. Limitaciones del sistema

Aunque el sistema cumple con el objetivo del proyecto, existen varias limitaciones.

- El lector está diseñado específicamente para imágenes PPM P3.
- El formato P3 utiliza texto, por lo que los archivos son considerablemente más grandes que formatos binarios.
- Cada región requiere iniciar una instancia independiente del intérprete Scheme.
- La comunicación Erlang–Scheme utiliza archivos temporales, lo que genera operaciones adicionales de entrada y salida.
- Las regiones completas deben serializarse a texto antes de enviarse a Scheme.
- El resultado debe convertirse nuevamente desde texto hacia las estructuras utilizadas por Erlang.
- La división actual se realiza únicamente mediante franjas horizontales.
- El rendimiento depende de la cantidad de núcleos disponibles y de la carga del equipo.
- Las pruebas fueron realizadas en una sola máquina, por lo que los tiempos absolutos no pueden generalizarse a otros equipos.
- El benchmark mide el sistema completo. Esto es útil para conocer el rendimiento real percibido, pero no permite conocer de manera aislada cuánto tiempo se utiliza únicamente dentro del filtro Scheme.

Una posible mejora futura sería reemplazar parte de la comunicación mediante archivos por un mecanismo de comunicación directa con el proceso Scheme.

También sería posible medir de forma separada los tiempos utilizados en lectura, división, procesamiento, comunicación, reconstrucción y escritura.

16. Conclusiones

El proyecto permitió integrar programación funcional y concurrencia utilizando dos lenguajes con responsabilidades diferentes.

Scheme se utilizó para implementar las transformaciones de imagen mediante recursión, listas y funciones, mientras que Erlang administró la lectura de las imágenes, división en regiones, creación de trabajadores, comunicación, tolerancia a fallos y reconstrucción.

El mecanismo de halo permitió que los filtros de convolución produjeran resultados correctos incluso en los límites entre regiones.

Las pruebas de rendimiento demostraron que la paralelización reduce de forma considerable el tiempo de procesamiento. En las cinco imágenes evaluadas, la configuración de ocho procesos presentó el menor tiempo absoluto.

Sin embargo, el rendimiento no crece indefinidamente de forma proporcional a la cantidad de procesos. Al pasar de cuatro a ocho procesos se observa una reducción de la eficiencia debido a costos como creación de procesos, ejecución de múltiples intérpretes Scheme, comunicación, archivos temporales, serialización, lectura, escritura y reconstrucción.

En promedio, la configuración de ocho procesos obtuvo un speedup aproximado de:

$$S_8 = 6,29$$

y una eficiencia aproximada de:

$$E_8 = 0,786$$

Los resultados muestran que el uso de Erlang permite paralelizar efectivamente el procesamiento de las regiones, pero también muestran la importancia de considerar los costos adicionales introducidos por la coordinación entre procesos y la comunicación entre lenguajes.

Finalmente, la implementación supera el filtro mínimo solicitado al incorporar múltiples transformaciones adicionales y soporte para Gaussian Blur con kernels de 3×3 y 5×5 .